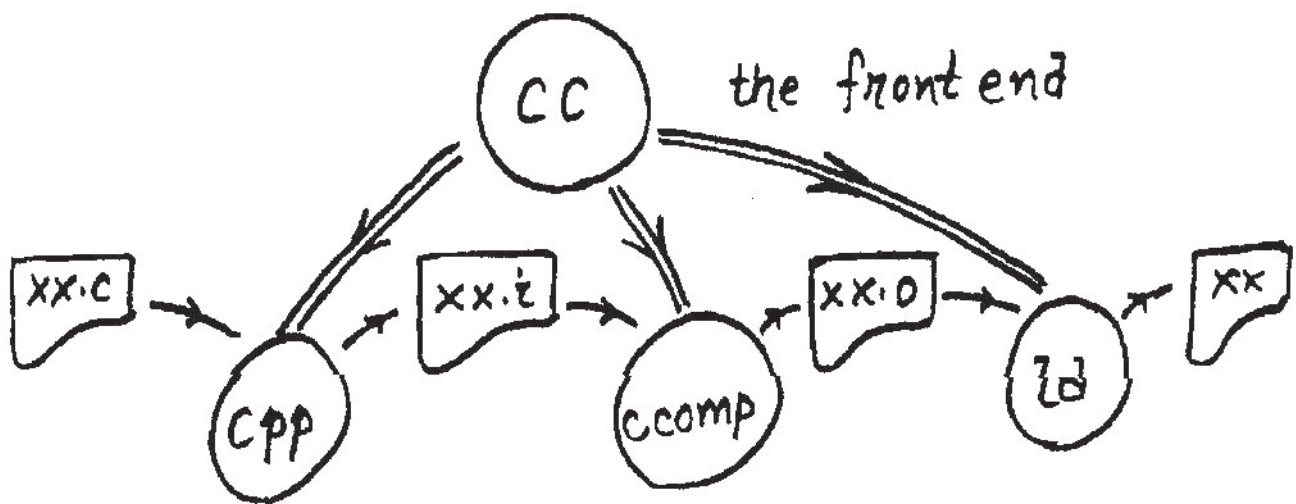


C FEATURES

Modularity ; Flexibility ; Space Vs. Time TradeOff
 Maintainability ; Documentability;
 Portability ; Testability



cpp , ccomp, ld : Which are responsible for which of the C features?

Modularity

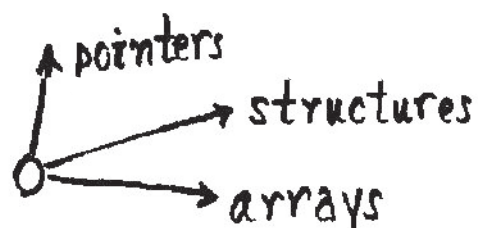
micro level (procedures)
 macro level (modules)

Responsible

: ccomp

: ld

Flexibility



: ccomp
 (C Syntax)

All Other Features: cpp is primarily responsible

C features

Space Vs. Time

Actual space OR time optimizations, by ccomp.
But tradeoffs for space to time OR
time to space, by CPP.

Maintainability

Extremely important
Bug Reports & changes of Specifications
are probable causes

Documentability

Needed for maintainability because of L.T.O.
(Labour Turn Over)

Self Documentation is superior to Comments.
An ART of programming

Portability

Needed for changes in environment

Ex: New CPU, New OS

Hardware obsolescence makes this serious.

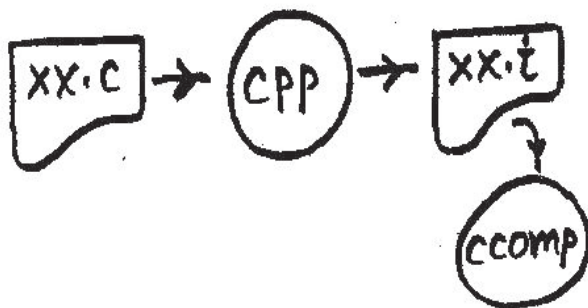
Testability : Ease with which testing can be done.

A Certification of Software.

Serious because of maintainability.

C PRE PROCESSOR

Q: What cpp must do to support the C features?



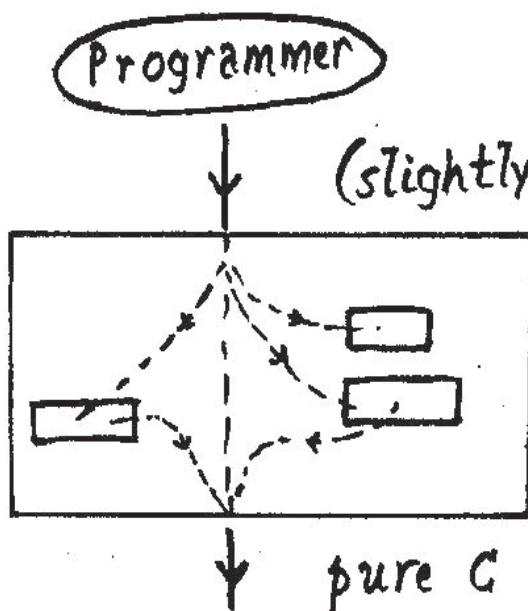
xx.i: 100% pure C syntax

xx.c: has impurities

cpp: is a translator

cpp: accepts "executable" or translatable statements
 accepts "declarative" or cpp Directives.

Approx. Block Diagram



A collection of datas
 : database of cpp.

Ex: "speak louder" affects further statements of the interpreter.

Ex: Symbol Table is database of C Compiler.

MAINTAINABILITY

A Code Fragment (from payroll application)

```

struct emp {
    ---
    double basic;
    ---
};
struct emp company[20];
/* to give an increment : Rs 20 */
---
int i;
for (i=0; i <= 19; i++)
    company[i].basic += 20;
---
```

Not interested in other members

procedure definitions not shown

$i < 20$ also ok

Changes of Specifications

- (a) No. of employees : say, changed to 22.
- (b) increment : say, changed to 2000.

Problem with Literals

- 20 : may mean different things
- 1996 : may be anything !

maintainability

Symbolic references have meaning

Ex: `int total;` is "some" total.

We need symbolic literals $\Rightarrow$ literals with names.

Another problem

is the number of changes to be made.

More changes $\Rightarrow$ Less confidence

Ex:

100 changes	:	Little confidence
10 changes	:	more confidence
2 changes	:	much better
1 change	:	probably best
0 changes	:	ideal !

This is "Assumed Lazyness" of programmer

SYMBOLIC LITERALS

We need

NEMP, INC

(usually all upper case letters)

cpp Directive: A "#" on, usually, column 1.

maintainability

Code Fragment

```
# define NEMP 20
# define INC 20
```

Name : important for
Self-documentation

just a set of characters

```
---
struct emp company[NEMP];
---
```

cpp substitutes
"20" for "NEMP"

```
for (i=0; i <= NEMP-1; i++)
    company[i].basic += INC;
```

Symbolic Literal Table

Name	String
NEMP	20
INC	20

NOTE: Code generated identical.
(20 - 1 computed by ccomp)

Simplicity of maintenance:
100's of changes effected automatically.

NOTE: Program to be recompiled if a change made.

Another Example

Look at the ternary operator ? :

maintainability

Code Fragment

```
int  x, y, z;
```

```
---
```

```
z = x > y ? x : y;      z = max. of x, y
```

```
---
```

```
z = x > y ? x : y;
```

```
---
```

```
z = y > y ? x : y;
```

(A mistake ! because of repetitions
No syntax error from compiler either)

Another example : name of the company

Use:

```
---  
printf ("Uttara");
```

```
#define CO_NAME "Uttara"
```

```
---  
printf ("Uttara");
```

```
---  
printf (CO_NAME);
```

```
---
```

```
---  
printf (CO_NAME);
```

NOTE: If symbolic literal definition is too long,
use:

\ <CR>

and continue definition to next line.

maintainability

max. code with symbolic literal

```
# define MAX    x > y ? x : y
```

```
--
```

```
z = MAX;        Avoid ; in the definition
```

```
--
```

```
z = MAX;
```

```
--
```

```
z = MIX;        Mistake! but compiler catches.
```

```
--
```

Now, we want to set x to max. of y, z .

$x = MAX;$ is wrong!

Reason: MAX is carrying wrong name.
Should have been called

MAX_ x - y

Now, we need a MAX_ y - z

```
# define MAX_y_z  y > z ? y : z
```

Q: How many MAX_... are we going to have?

A: Parameterize the definition

We need: MAX (<something>, <somethingelse>)

This is called a MACRO.

MACROS

Code Fragment

```
# define max(a, b)    a > b ? a : b
```

no gap

a, b: dummy "variables"

```
-- --  
int x, y, z;
```

We used lowercase
letters for max

```
-- --  
z = max(x, y);
```

```
-- --  
x = max(y, z);
```

Hence, max(x, y)

```
-- --  
y = max(x, z);
```

looks like procedure invocation

The code fragment works if we replace
the macro definition by a procedure:

```
int max(a, b) int a, b; {  
    return a > b ? a : b;  
}
```

Q: Datatype of returned value?
Datatypes of parameters?

Ex: max(12.3, 13.4)

may produce wrong results.

(MACROS Vs. PROCEDURES)

We have a choice:

Use macro definition OR procedure definition.

Implies:

Sometimes use macros

Othertimes use procedures

Macro

inline expansion

(do it yourself)

No parameter passing

Procedure

invoke procedure

(delegate responsibility)

Independantly compitable

Two Jobs

dig a borewell

usually delegated;

Complicated Job

wipe my forehead

usually, do it yourself

Simple Job

(Parameter passing may
take longer than the job)

Q: How to decide a job is simple OR complex?

A: An engineering decision to be made

macros vs. procedures

datatype insensitive

Ex:

```
double p, q, r;
```

```
p = max(q, r);
```

works!

datatype sensitive

"max" carrying wrong name

Should be:

```
max_int_int_int
```

We may also need:

```
max_double_double_double,  
etc.
```

Decision of macro/procedure only if datatypes same.

Trade Offs: Program Size Vs. Execution Speed.

JOB

— > — ? — : —

Size: S_{BODY} bytes of instructions.

Time: T_{BODY} μsec . to execute these.

INVOKATIONS

for macros OR procedures

N_{PHYS} : physical occurrences of invocations

N_{LOGICAL} : CPU executes the invocation.

Ex: for ($i=0$; $i < 10000$; $i++$)

```
z = max(x, y);
```

macros vs. procedures

Use Macro Definitions

$$\text{Code Size} = S_{\text{BODY}} * N_{\text{PHYS}} \text{ bytes}$$

$$\text{Exec. Time} = T_{\text{BODY}} * N_{\text{LOGICAL}} \text{ } \mu\text{sec.}$$

Use Procedure Definition

Body of Code has : preamble & postamble

Each invocation : pass parameters & invoke

S_{AMBLE} bytes : preamble + postamble code

T_{AMBLE} $\mu\text{sec.}$: exec. time for this.

S_{INVOKE} bytes : invocation overhead

T_{INVOKE} μsec : exec. time for this

$$\text{Code Size} = S_{\text{BODY}} + S_{\text{AMBLE}} + S_{\text{INVOKE}} * N_{\text{PHYS}} \text{ bytes}$$

$$\text{Exec. Time} = N_{\text{LOGICAL}} * (T_{\text{BODY}} + T_{\text{INVOKE}} + T_{\text{AMBLE}}) \text{ } \mu\text{sec}$$

HENCE

- (a) Macros are always faster.
- (b) Usually procedures are compact
- (c) For simple jobs, macros are more compact also.